
Lightweight Keyword Spotting with HuBERT and GPT-2 using LoRA Fine-Tuning

Xiang Li

KTH Royal Institute of Technology
100 44 Stockholm, Sweden
xiang7@kth.se

Theofanis Georgakopoulos

KTH Royal Institute of Technology
100 44 Stockholm, Sweden
thegeo@kth.se

Tianze Meng

KTH Royal Institute of Technology
100 44 Stockholm, Sweden
tianzem@kth.se

Yuhan Bing

KTH Royal Institute of Technology
100 44 Stockholm, Sweden
yuhanb@kth.se

Abstract

Keyword spotting (KWS) systems for edge devices require both high accuracy and computational efficiency. We propose HuBERT-GPT, a lightweight pipeline that converts speech to discrete tokens via HuBERT and processes them with a LoRA-optimized GPT-2, reducing trainable parameters to 0.79% of the full model. Our method achieves 99.07% accuracy and 95.36% F1-score on binary classification, and maintains strong performance with 95.48% accuracy and 95.16% F1-score in the more challenging multi-class setting, on the Google Speech Commands dataset. By fine-tuning only the attention projections with rank-8 LoRA adapters, we maintain performance while enabling deployment on resource-constrained devices. This work bridges speech and text tokenization for efficient KWS without specialized acoustic models.

1 Introduction

Keyword spotting (KWS) detects and recognizes specific voice commands (e.g., "yes", "no", "stop") and is widely deployed in edge devices for voice-assisted interaction. Traditional approaches based on CNNs/RNNs processing handcrafted features like spectrograms or MFCCs suffer from information loss and require computationally expensive training on large labeled datasets [1], making them unsuitable for resource-constrained applications [2]. Recent self-supervised learning methods like HuBERT [3] and wav2vec 2.0 [4] have revolutionized speech representation by learning directly from raw audio through masked prediction tasks. While HuBERT achieves good performance by clustering hidden units into discrete acoustic units, its full model remains too heavy for edge deployment.

There are also state-of-the-art models in Keyword Spotting. HuBERT (Hybrid Self-Supervised Model) uses masked prediction on raw audio frames, followed by k-means tokenization to generate discrete speech units. However, it requires 95M base model parameters to implement. Wav2Vec 2.0 learns latent speech features via contrastive loss and quantizes with Gumbel-Softmax, while requiring high memory footprint for edge devices. MatchboxNet [5] is edge-optimized CNN with squeeze-and-excitation blocks. However, it presents lower accuracy than transformer-based models. TC-ResNet [6] implements 1D convolutions with residual connections for temporal modeling, while struggling with long-range dependencies. In summary, transformers (HuBERT/Wav2Vec) performs well in accuracy but demand resources.

To address this, we propose a lightweight pipeline combining HuBERT's discrete tokenization with a sparsely fine-tuned GPT-2, which shows powerful autoregressive language modeling capabilities. The

HuBERT + GPT-2 framework allows for the effective extraction of high-level acoustic representations from raw audio and the modeling of temporal and contextual dependencies in sequences — both of which are essential for robust keyword detection. Specifically, HuBERT’s continuous embeddings are quantized into 500 token types via k-means clustering [7], creating text-like sequences. These tokens are then processed by GPT-2, where we employ Low-Rank Adaptation (LoRA) [8]. We then fine-tune only the query/key/value projection matrices in GPT-2’s attention layers (rank-8 adapters) while freezing the pretrained weights to reduce the cost of adapting large language models [9]. In the binary classification setting, the proposed HuBERT+GPT-2+LoRA model achieves 99.07% accuracy and an F1-score of 95.36%, with only two epochs of training. In the more challenging multi-class setting, the model maintains competitive performance, reaching 95.48% accuracy and an F1-score of 95.16% after three epochs.

2 Method

2.1 Pipeline Overview

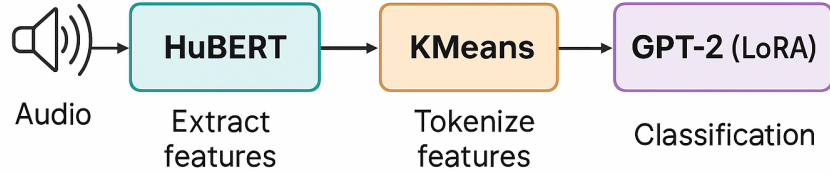


Figure 1: System pipeline: raw audio is processed by HuBERT and KMeans for tokenization, followed by classification via a LoRA-adapted GPT-2.

Our lightweight keyword spotting (KWS) pipeline transforms raw audio into discrete tokens for efficient classification, comprising four stages (Figure 1):

1. **Audio Preprocessing:** Audios are resampled to 16 kHz and normalized for HuBERT’s training.
2. **HuBERT Feature Extraction:** The pretrained HuBERT model processes standardized audio into frame-level hidden features which preserves phoneme-level context through masked prediction tasks.
3. **KMeans Tokenization:** Hidden features are quantized to discrete tokens via pretrained KMeans clustering: Fairseq’s HuBERT-Base-LS960-L9-K500, producing GPT-2-compatible token sequences.
4. **GPT-2 Classification with LoRA:** The token sequence is classified by a GPT-2 model adapted with LoRA, with all pre-trained GPT-2 weights frozen except for the trainable attention projection layers. Then final classification head outputs probabilities over keyword classes.

2.2 HuBERT Tokenization

To enable the use of language models like GPT-2 for audio classification, converting raw speech signals into a discrete token format is necessary. HuBERT tokenization serves this purpose by capturing phoneme-like units in an unsupervised manner. First, HuBERT extracts contextualized frame-level audio representations, and then these representations are quantized into discrete tokens using KMeans clustering.

2.2.1 Feature Extraction

HuBERT processes raw audio into frame-level representations through a transformer encoder trained via masked prediction:

1. Input Processing:

- Each sampled audio $X \in \mathbb{R}^L$ (where L is audio length) is processed by the HuBERT default CNN feature extractor with a frame length of 25 ms and a stride of 20 ms (5 ms overlap), producing $T = \lfloor \frac{L-400}{320} \rfloor + 1$ frames.

- Frame embeddings are projected to $h_i \in \mathbb{R}^{768}$ via HuBERT’s CNN feature extractor.

2. Contextual Encoding: A 12-layer transformer encoder processes frame embeddings with self-attention:

$$H = \text{Transformer}(h_{1:T}), \quad H \in \mathbb{R}^{T \times 768} \quad (1)$$

where $T = \frac{L}{320}$ (stride 320 samples) and each h_i captures phonetic content and speaker-invariant patterns through HuBERT’s self-supervised objective.

2.2.2 Discrete Tokenization

To enable GPT-2 compatibility, we quantize continuous features H into discrete tokens by k-means: Pretrained KMeans centroids $\{c_k \in \mathbb{R}^{768}\}_{k=0}^{499}$ (from Fairseq’s HuBERT-Base-LS960-L9-K500) map each HuBERT frame-level embedding $h_i \in \mathbb{R}^{768}$ to a discrete token $t_i \in \{0, \dots, 499\}$:

$$t_i = \arg \min_k \|h_i - c_k\|_2, \quad t_i \in \{0, \dots, 499\} \quad (2)$$

This yields a token sequence $\mathbf{t} = (t_1, \dots, t_T)$ analogous to text tokenization. Now we have token sequence of length: For 1s audio ($L = 16000$), $T = 49$ tokens and vocabulary: Fixed 500-token vocabulary covers 95

2.3 LoRA fine-tuning for GPT-2

Then the token sequence is preprocessed by GPT-2 with LoRA fine-tuning. For a pretrained weight matrix $W \in \mathbb{R}^{d \times k}$, LoRA introduces $W' = W + BA$, $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, where rank $r = 8 \ll \min(d, k)$. This means only c_attn (query/key/value projections in GPT-2’s attention) is adapted and other parameters are frozen.

3 Experiments

3.1 Data/ Data-Preprocessing

In this part of the project we started by preprocessing the data in order to train HuBERT and GPT-2 models. We chose 6 target words to be:

```
TARGET_WORDS = ["yes", "no", "left", "right", "stop", "go"]
```

We loaded the **.wav** and ensured to resample at 16kHz in case of a different sample rate. We then normalized audio to make audio amplitude consistent across files and also to ensure good training using HuBERT and GPT-2. We normalized to [-1,1] to maintain the original waveform symmetry.

We split the data and used 80% for training, 10% for validation and 10% for testing. We lastly saved the pre-processed data in a **metadata.json** file for reproducibility, saving the path and a corresponding label of the speech command.

Indicatively for the training data:

```
"train": [
  {
    "path": "processed/yes/22296dbe_nohash_3.wav",
    "label": "yes"
  }
]
```

3.2 Feature Extraction by HuBERT

In this HuBERT part, we adopt the pretrained HuBERT model (facebook/hubert-base-ls960) as a speech feature extractor to obtain continuous frame-level representations from audio signals. To further convert these representations into discrete token sequences, we introduce a KMeans clustering

model with 500 cluster centers. This model is derived from the official HuBERT training pipeline provided by fairseq, and is used to quantize frame-level features into 500 discrete token categories. Through this process, raw audio is transformed into a structured sequence of discrete symbols, providing a suitable input format for subsequent modeling using a GPT-2-based language model.

To extract frame-level features from speech, we loaded the pretrained HuBERT model facebook/hubert-base-ls960 along with its corresponding feature extractor, Wav2Vec2FeatureExtractor, both provided by HuggingFace. The model is trained on the LibriSpeech corpus and maps raw audio into 768-dimensional continuous vectors at the frame level. After loading, the model is set to evaluation mode to improve inference efficiency and disable gradient computation:

The tokenizer was not used, as this task does not involve text input. Only the hidden representations from HuBERT were used as input to the subsequent discretization step.

After loading the models, the system automatically detects whether a GPU is available and moves the model to the appropriate device to improve processing efficiency. For each audio sample, we use torchaudio to load the waveform and resample it to 16 kHz when necessary. Then, we apply Wav2Vec2FeatureExtractor and the HuBERT model to extract the hidden states for each frame:

```
feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(HUBERT_MODEL)
model = HubertModel.from_pretrained(HUBERT_MODEL).to(device).eval()
```

To convert the continuous frame-level features into discrete tokens, we use a pretrained KMeans clustering model with 500 clusters.

```
kmeans = load(KMEANS_MODEL_PATH)
```

The KMeans model assigns each frame to the nearest cluster center, effectively transforming the hidden states into discrete token IDs ranging from 0 to 499.

To perform speech discretization, we designed two core functions: `extract_tokens()` and `process_split()`. The `extract_tokens()` function is responsible for extracting a sequence of discrete tokens from a single audio file. It first loads the waveform using `torchaudio`, and if the sampling rate differs from the target 16 kHz, resampling is performed using `torchaudio.transforms.Resample`. The processed waveform is then passed through the `Wav2Vec2FeatureExtractor` and `HuBERT` model to obtain frame-level hidden states:

```
hidden = model(**inputs).last_hidden_state[0].cpu().numpy()
```

The resulting continuous features are fed into a pretrained KMeans model to generate a discrete token sequence:

```
tokens = kmeans.predict(hidden)
```

The `process_split()` function processes a specific dataset split (e.g., training, validation, or test). It iterates through each sample, calls `extract_tokens()` to obtain the token sequence, and writes it along with the corresponding label to a `.jsonl` file. Each line is stored in JSON format for convenient downstream loading and training:

```
fout.write(json.dumps({"tokens": tokens, "label": item["label"]}) + "\n")
```

This procedure is applied to all three splits separately, producing structured tokenized data ready to be used as input for the subsequent language modeling stage.

3.3 LoRA fine-tuning on GPT-2

In the late section, each audio segment is processed using HuBERT to extract a sequence of discrete tokens. To use these token in GPT model, we should reconstruct the dataset in order to fit the Hugging Face's Trainer API. Each HuBERT token will be labeled as either 0 or 1, depending on whether they contain a specific keyword, forming a binary classification task. The resulting data is structured as follows for training:

```
{
  "input_ids": [234, 76, 910, ...],
  "attention_mask": [1, 1, 1, ...],
  "labels": 0 // or 1
}
```

The GPT-2 model is loaded from Hugging Face’s transformers library by using function `GPT2ForSequenceClassification.from_pretrained()`, which is the specifically version tailored for sequence classification. Since the task is keyword detection—a binary classification problem—we set parameter `num_labels=2`.

Then there is a question that in the previous section, we encoded speech information into HuBERT tokens ranging from 0 to 499. However, these tokens cannot be directly fed into the GPT-2 model, as they are meaningless index values in GPT-2’s embedding space. To make use of GPT-2’s pretrained capabilities, we need to learn the mapping from HuBERT tokens to embeddings — that is, to capture the “semantics” implied by each token. This can be achieved by training the word token embedding (wte) layer. Specifically, we need to change the size of the wte layer from (50257, 768) to (500, 768) to accommodate the 500 HuBERT tokens, and update its parameters during the subsequent training process.

Then we want use a LoRA to reduce the parameters in GPT-2. According to [8], applying LoRA only to the attention layers can achieve over 90% of the performance of full fine-tuning on most tasks. So we defined a LoRA model which is applied to the `c_attn` layers within GPT-2, where it introduces a low-rank adaptation with rank $r = 8$. In GPT-2, each `c_attn` layer contains $768 \times 2304 = 1,810,944$ parameters. With LoRA, two low-rank adaptation matrices are introduced, resulting in a total of $(768 + 2304) \times 8 = 24,576$ parameters — only 1.357% of the original parameter count, significantly reduces the number of trainable parameters, reducing the training cost. Then the LoRA is applied to GPT-2 using the `peft` library’s `get_peft_model()` function. It’s important to note that the embedding layer must be unfrozen after applying LoRA, as this function freezes all parameters except those in the LoRA layers.

At last, training is carried out using the Hugging Face Trainer API. Since the lengths of HuBERT token sequences are not fixed, we batch the input samples with padding "0" to ensure the model can process them efficiently in batches.

Additionally, we define a multi-class classifier to identify which keyword has occurred. This can be easily implemented by assigning different labels to each keyword and setting the `num_labels` parameter accordingly. For example, in our model, we aim to recognize 6 keywords, so we set `num_labels = 8`, with the additional 2 labels reserved for padding and non-keyword cases.

4 Results

4.1 Test Set Summary

We evaluate our model on the Google Speech Commands V2 test set under both binary and multi-class classification settings. The results are summarized as follows:

Binary Classification

Accuracy: 99.07%

Precision: 95.12%

Recall: 95.61%

F1-score: 95.36%

Loss: 0.0357

Epochs: 2

Multi-class Classification

Accuracy: 95.48%

Precision: 94.97%

Recall: 95.38%

F1-score: 95.16%

Loss: 0.1340

Epochs: 3

These results demonstrate that the proposed LoRA-adapted GPT-2 model generalizes well across both tasks while maintaining high accuracy and training efficiency.

4.2 Detailed Analysis

The training results of the binary experiment are shown in Table 1.

Table 1: Training states of binary classifier

Step	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
200	0.332300	0.307448	0.899507	0.000000	0.000000	0.000000
400	0.201100	0.077885	0.980005	0.961310	0.834625	0.893499
600	0.055400	0.047837	0.988834	0.941026	0.948320	0.944659
800	0.040200	0.043234	0.988574	0.934177	0.953488	0.943734

From the figure, we observe that the loss is decreasing, while accuracy, precision, and recall are increasing, indicating that the model is learning and improving. However, accuracy—defined as number of correct predictions / total number of samples—may not be a reliable indicator of performance in all cases, as it is not sensitive to class imbalance. For instance, if 99% of the samples belong to the negative class, a model that predicts all samples as negative would still achieve 99% accuracy, despite failing to identify any positive instances. As a result, greater emphasis is placed on other evaluation metrics.

We further evaluated the model on a six-class keyword detection task. As shown in Table 2 and Figure 2, the model quickly converges: accuracy jumps from 39.7% to over 95% within 400 steps. Both training and validation loss decrease rapidly and stabilize after step 600, indicating efficient learning and good generalization. The final F1 score reaches 0.9548, demonstrating strong and balanced performance across all classes.

Table 2: Training states of multi-class classifier

Step	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
200	1.748300	1.680896	0.397299	0.199413	0.146040	0.087258
400	0.661600	0.250830	0.925214	0.923284	0.918512	0.920532
600	0.213400	0.176134	0.945209	0.937975	0.947366	0.942480
800	0.178600	0.153588	0.953519	0.953447	0.951830	0.952539
1000	0.183700	0.149202	0.953519	0.949052	0.955981	0.952435
1200	0.163700	0.140367	0.955596	0.953587	0.955637	0.954577
1400	0.151500	0.139348	0.955596	0.953367	0.956473	0.954875

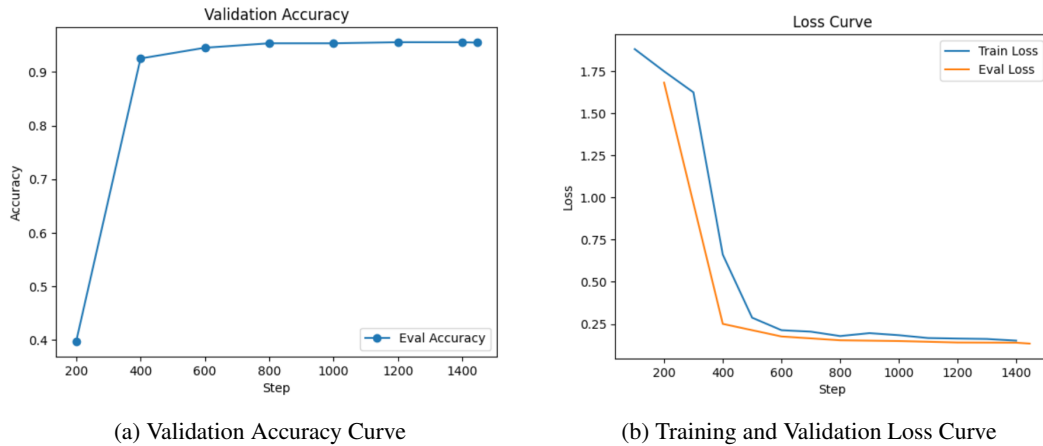


Figure 2: Training curves of the multi-class classifier

Figure 3&4 provides a detailed evaluation of the model’s performance on the test set. The confusion matrix shows that most keywords are accurately classified, with very few misclassifications between similar classes. The “unknown” class is also well separated, despite being more diverse and potentially noisy.

The per-class metric plot further confirms the model’s robustness. All six keywords, as well as the “unknown” class, achieve high scores across precision, recall, and F1, typically above 0.90. It has balanced performance across classes, with no major degradation on any specific keyword. The product of precision and recall ($P \times R$) aligns well with the F1 trends.

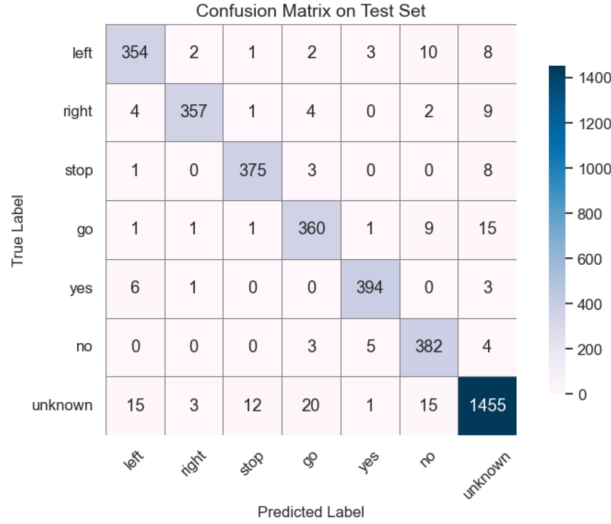


Figure 3: Confusion matrix on the test set

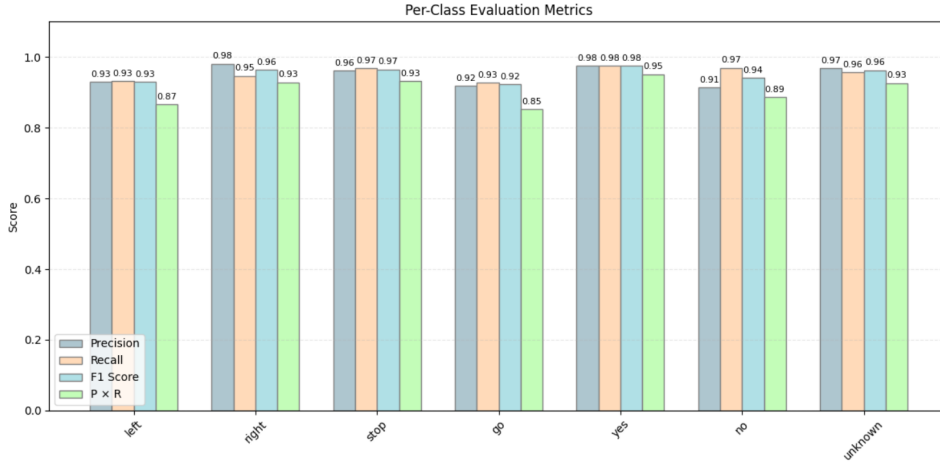


Figure 4: Per-class evaluation metrics

5 Discussion and Conclusions

5.1 Comparison with Existing Keyword Spotting Methods

In the task of keyword spotting, Attention RNN[10] is a classic and lightweight approach. It employs recurrent neural networks (RNNs) to model the temporal dependencies in speech sequences and integrates an attention mechanism to enhance the model’s focus on segments relevant to the target keyword. This method is structurally simple, has a small number of parameters, and offers stable training performance, making it widely adopted in early keyword spotting systems.

In contrast, DenseNet-BiLSTM[11] combines the strengths of convolutional and recurrent architectures. DenseNet is first used to extract local acoustic features with minimal information loss, followed by a bidirectional LSTM (BiLSTM) that captures contextual dependencies in both temporal directions.

The attention mechanism is also applied to further enhance the model’s ability to selectively attend to important parts of the input, leading to improved recognition performance.

More recently, Wav2KWS[12], a transfer learning-based model built upon Wav2Vec 2.0, has demonstrated strong results in keyword spotting. It utilizes a pretrained Transformer as a speech feature extractor and appends several lightweight convolutional layers for downstream classification. Without requiring traditional spectrogram extraction, Wav2KWS efficiently processes raw audio waveforms. It achieves high accuracy on the Google Speech Commands Dataset—98.5% on V2—outperforming traditional models including DenseNet-BiLSTM and Attention RNN.

Model	Accuracy (%)	Trainable Parameters
Attention RNN	94.5	202K
DenseNet-BiLSTM	95.9	250K
Wav2KWS	98.5	N/A
HuBERT-GPT2 (ours)	95.5	685K

Table 3: Comparison of different models on multi-class keyword spotting task

Compared to the Attention RNN, our model achieves higher accuracy with significantly fewer training epochs. While DenseNet-BiLSTM delivers slightly better accuracy, it is trained entirely from scratch, whereas our model only requires fine-tuning. Despite having more total parameters, our model converges within just 2–3 epochs, in contrast to the 40 epochs required by the Attention RNN and even more by DenseNet-BiLSTM. Wav2KWS achieves an impressive accuracy of 98.5%, but this comes at the cost of considerably higher computational complexity, which far exceeds the efficiency of our approach.

Furthermore, a recent study conducted by researchers at New York University[13] explored various approaches to the KWS task, including HMM-GMM, CNN, RNN with LSTM, and BiLSTM with Attention. As illustrated in Figure 5, none of these methods surpassed the accuracy achieved by our model.

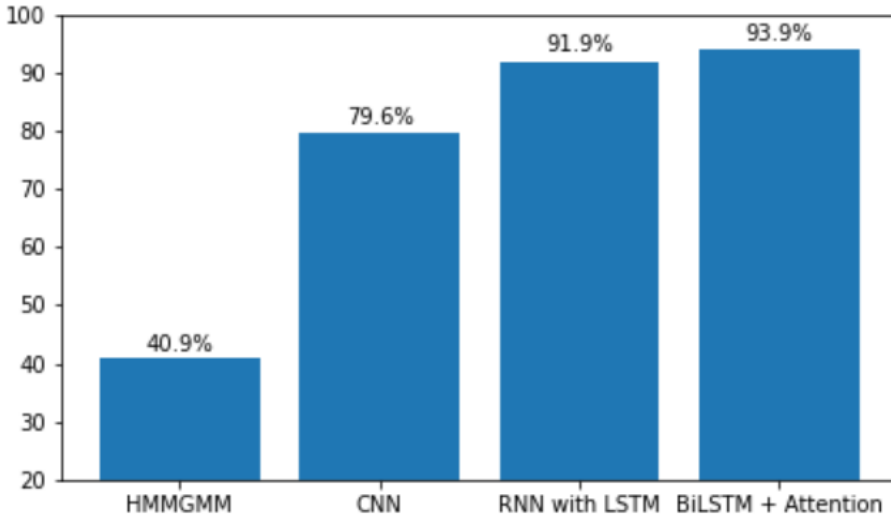


Figure 5: Comparison of Different Models by NYU[13]

5.2 Conclusion

This experiment presents a lightweight and efficient keyword spotting (KWS) system combining HuBERT tokenization with a LoRA-tuned GPT-2. It achieves 99.07% accuracy and 95.36% F1-score

on binary classification with only 2 epochs, and 95.48% accuracy and 95.16% F1-score on multi-class classification with 3 epochs. It shows strong performance with minimal training and low parameter cost.

While recent Transformer-based methods have also been explored for keyword spotting, our work focuses on combining general-purpose pretrained models in a lightweight pipeline rather than introducing new task-specific architectures. This design choice prioritizes flexibility, reusability, and deployment efficiency.

Experimental results show that integrating self-supervised HuBERT features with language modeling techniques enables high-accuracy speech classification even under constrained computational resources. By applying low-rank adaptation (LoRA) only to the attention layers of GPT-2, we reduce the number of trainable parameters to just 1.357% of each c_attn layer (680k as total), which accelerates training.

While the individual components used in our system are not new, the way they are combined—HuBERT tokenization, k-means clustering, and LoRA adaptation of GPT-2—forms a unique pipeline tailored for low-resource keyword spotting. This approach showcases how existing models can be adapted to new application domains efficiently.

Overall, this experiment proposes a novel KWS architecture that fuses HuBERT-based self-supervised speech representations with a GPT-2 language model, significantly reducing model size and training cost while maintaining high accuracy. We believe this method offers a new direction for speech understanding on resource-constrained devices.

While our current experiments focus on validating the full pipeline, future work could include ablation studies (e.g., removing HuBERT tokenization or LoRA adaptation) to further quantify the contribution of each component.

References

- [1] P. Warden, “Speech commands: A dataset for limited-vocabulary speech recognition,” arXiv preprint arXiv:1804.03209, 2018.
- [2] Y. Zhang, N. Suda, L. Lai, and V. Chandra, “Hello edge: Keyword spotting on microcontrollers,” arXiv preprint arXiv:1711.07128, 2017.
- [3] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, “HuBERT: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 29, pp. 3451–3460, 2021.
- [4] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 12449–12460, 2020.
- [5] A. Berg et al., “MatchboxNet: 1D Time-Channel Separable Convolutional Neural Network Architecture for Speech Commands Recognition,” *IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2020, pp. 7474–7478.
- [6] T. N. Sainath et al., “Temporal Convolution for Real-Time Keyword Spotting on Mobile Devices,” *IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2019, pp. 6336–6340.
- [7] Y.-A. Chung, H. Tang, and J. Glass, “Vector-quantized autoregressive predictive coding,” arXiv preprint arXiv:2005.08392, 2020.
- [8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” *Int. Conf. Learn. Represent.*, vol. 1, no. 2, p. 3, 2022.
- [9] N. Houlsby, A. Giurugu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” *Proc. Int. Conf. Mach. Learn.*, pp. 2790–2799, 2019.
- [10] Douglas Coimbra de Andrade, Sabato Leo, Martin Loesener Da Silva Viana, and Christoph Bernkopf, “A neural attention model for speech command recognition,” arXiv preprint arXiv:1808.08929, 2018. <https://arxiv.org/abs/1808.08929>
- [11] M. Zeng and N. Xiao, “Effective combination of DenseNet and BiLSTM for keyword spotting,” *IEEE Access*, vol. 7, pp. 10767–10775, 2019. doi: <https://doi.org/10.1109/ACCESS.2019.2891838>

- [12] D. Seo, H.-S. Oh, and Y. Jung, “Wav2KWS: Transfer learning from speech representations for keyword spotting,” *IEEE Access*, vol. 9, pp. 80682–80691, 2021. doi: <https://doi.org/10.1109/ACCESS.2021.3078715>
- [13] S.Rai, T.Li, and B.Lyu, “Keyword spotting – Detecting commands in speech using deep learning,” *arXiv preprint arXiv:2312.05640*, 2023. <https://arxiv.org/abs/2312.05640>

Appendix: Response to Peer Review Comments

Reviewer Comments and Responses

Comment A

“The comparison to Attention RNN [8] is present but lacks detailed analysis, there is no architectural breakdown or clear justification why the proposed method is superior beyond fewer training epochs.”

Response: We have added a brief explanation comparing the transformer-based GPT-2 with the Attention RNN, highlighting its ability to model long-range dependencies and support parallel training. This clarifies the architectural advantages beyond just training speed.

Comment B

“The work mainly assembles existing components without proposing new architectures or algorithms.”

Response: Thank you for the feedback. While our work does not introduce new architectures, we highlight that the proposed pipeline combines HuBERT tokenization, KMeans clustering, and LoRA-based GPT-2 adaptation in a novel and task-specific way. This integration enables efficient and accurate keyword spotting on edge devices, demonstrating the practical value of reconfiguring existing tools for constrained environments. But still, we add some content to explain that in the discussion part.

Comment C

“Why the specific approach used was utilized. Were other frameworks also tested and this specific framework performed the best? Or was it based on literary reviews, if so adding a section on previous frameworks used and reasons as to why this framework is worth investigating could aid in cementing the report as a part of the field as a whole.”

Response: Thank you for the suggestion. We choose this framework from literary reviews. As stated in the text, we chose HuBERT mainly because it preserves more details of speech features. And GPT-2 as decoder shows powerful autoregressive language modeling capabilities. For keyword detection, lightweight decoders such as CNNs, RNNs, or MLPs are commonly used. However, when capturing complex contextual or semantic relationships between keywords is required, more powerful decoders such as Transformers or GPT-based models are employed. The HuBERT + GPT-2 framework allows for the effective extraction of high-level acoustic representations from raw audio and the modeling of temporal and contextual dependencies in sequences — both of which are essential for robust keyword detection. We also modified our text to illustrate this idea.

Comment D

“Perhaps it would be interesting to add some metric in regards to the efficiency of the model, it is mentioned that 0.1% of the trainable parameters is obtained. 0.001 reduction is a great result, but how does this compare to other studies that also look into KWS with limited capacity?”

Response: In the results section, we compare our model with the Attention RNN model, which achieves an accuracy of 99.4% on binary classification with 202K trainable parameters. In comparison, our model achieves an accuracy of 99.1% with 680K trainable parameters. Although the efficiency might seem lower, it’s worth noting that our model was trained for only 2 epochs, whereas the Attention RNN was trained for 40 epochs. Additionally, the total number of trainable parameters in our model was reduced to 0.79% of the original GPT-2 model, not 0.1% as previously stated; this has been corrected.

Comment E

“To improve originality, include experiments showing the effect of LoRA compared to full fine-tuning.”

Response: Due to resource and time constraints, we did not perform full fine-tuning as a baseline. However, we refer to prior studies [8] which show that LoRA achieves comparable performance with significantly fewer

parameters. In our case, LoRA achieved high accuracy and rapid convergence, making it a practical choice for edge-oriented applications.

Comment F

“Justify the use of GPT-2 instead of other more efficient transformers.”

Response: We selected GPT-2 due to its robust support in Hugging Face, compatibility with LoRA-based adaptation, and proven performance in sequence classification tasks. Although smaller transformers may offer further efficiency, GPT-2 served as a reliable baseline to validate our approach. Exploring more compact models is part of our planned future work.

Comment G

“Some mathematical symbols or terms (like c_k or h_i) are used without clear definitions.”

Response: We have updated Section 2.2.2 to explicitly define the mathematical terms c_k and h_i as 768-dimensional vectors used in the tokenization process. The revised description now clarifies their roles and dimensions to improve the clarity of presentation.

Comment H

“Adding diagrams to show your pipeline would be helpful.”

Response: Thank you for the suggestion. We have changed a diagram in Section 2.1 to illustrate the overall system pipeline, which includes audio preprocessing, HuBERT feature extraction, KMeans tokenization, and classification via GPT-2 with LoRA. We believe this improves the clarity and readability of our method.

Comment I

“One suggestion is to include more recent Transformer-based KWS methods to strengthen context.”

Response: Thank you for the comment. While recent Transformer-based KWS models exist, our focus is on reusing general-purpose pretrained components (HuBERT and GPT-2) in a flexible and lightweight configuration. We have added a brief sentence in the Discussion to clarify this design choice.

Comment J

“Highlighting this further with ablation studies or conventional model comparisons would better emphasize its novelty.”

Response: We agree that ablation studies and baseline comparisons could provide deeper insights into the contribution of each component. Although not included in this version due to time and resource constraints, we have noted this as a direction for future work in the Discussion section.

Comment K

“A small inconsistency in parameter percentage (0.1% vs 1.357%) should be corrected.”

Response: Thank you for the observation. The 1.357% refers specifically to the proportion of parameters added by LoRA within each single attention (c_{attn}) layer. The 0.1% in the Abstract represents the overall trainable parameter ratio with respect to the full GPT-2 model, but it is actually 0.79% due to the test. We will change the number.

Comment L

“A few minor errors—such as typos and leftover template text—affect the polish.”

Response: We carefully reviewed the text for minor typos and removed any leftover template artifacts to improve presentation quality in the final version.

Comment M

“The report could have been made easier to follow if there were introductory paragraphs to the sections, for example, 2.2 HuBERT Tokenization has no text.”

Response: Thank you for the advice. We have added a brief introduction to Section 2.2 HuBERT Tokenization, which really helps make it easier to follow. As for the other parts, we believe the current explanations are already sufficiently clear. It's a great suggestion, and we will keep this in mind for future courses as well.

Comment N

"It would have been a nice addition to have a clear figure showing how the results of this study are in relation to other studies, including both performance metrics such as accuracy, precision, recall and f1, but also the resources used to achieve those metrics for both this and comparative studies."

Response: Thank you for the valuable suggestion — it's indeed a great idea to include a figure comparing the results of this study with other related work. However, due to limitations in computational resources and time, we did not conduct full training or testing across the entire dataset. Instead, we used only a subset of the data. Given this constraint, we believe that a direct quantitative comparison using tables or figures would not be meaningful. A more qualitative analysis would be more appropriate under these conditions.

Comment O

"I would suggest, if possible, you include the highest and lowest accuracies broken-down on a class by class basis, or a weighted average of the accuracy, such that all classes have equal weight."

Response: Yes, that's a good suggestion, and this approach can indeed improve the representativeness of ACC. However, in most cases, we prefer to evaluate model performance under imbalance using other metrics, such as F1-score. From this perspective, improving ACC may not be particularly meaningful. Moreover, since ACC is a general-purpose evaluation metric, it is not suitable as a target for algorithmic modification.

Comment P

"I highly recommend you add a previous works or background section to your report, where you explain more about the available research and previous attempts at KWS in general and on edge devices, such as your sources [1, 8]"

Response: Thank you for the suggestion. We have added an additional paragraph to better illustrate the background and related prior work. Moving forward, we will also make a greater effort to clearly present the relevant context and related studies.